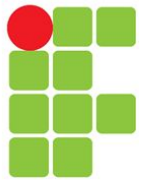


INSTITUTO FEDERAL
SÃO PAULO



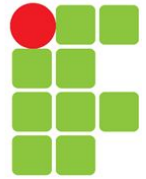
WebSocket e Protocolo STOMP

Desenvolvimento Web Backend (BRADWBK)

Prof. Luiz Gustavo Diniz de Oliveira Véras

E-mail: gustavo_veras@ifsp.edu.br

Objetivos



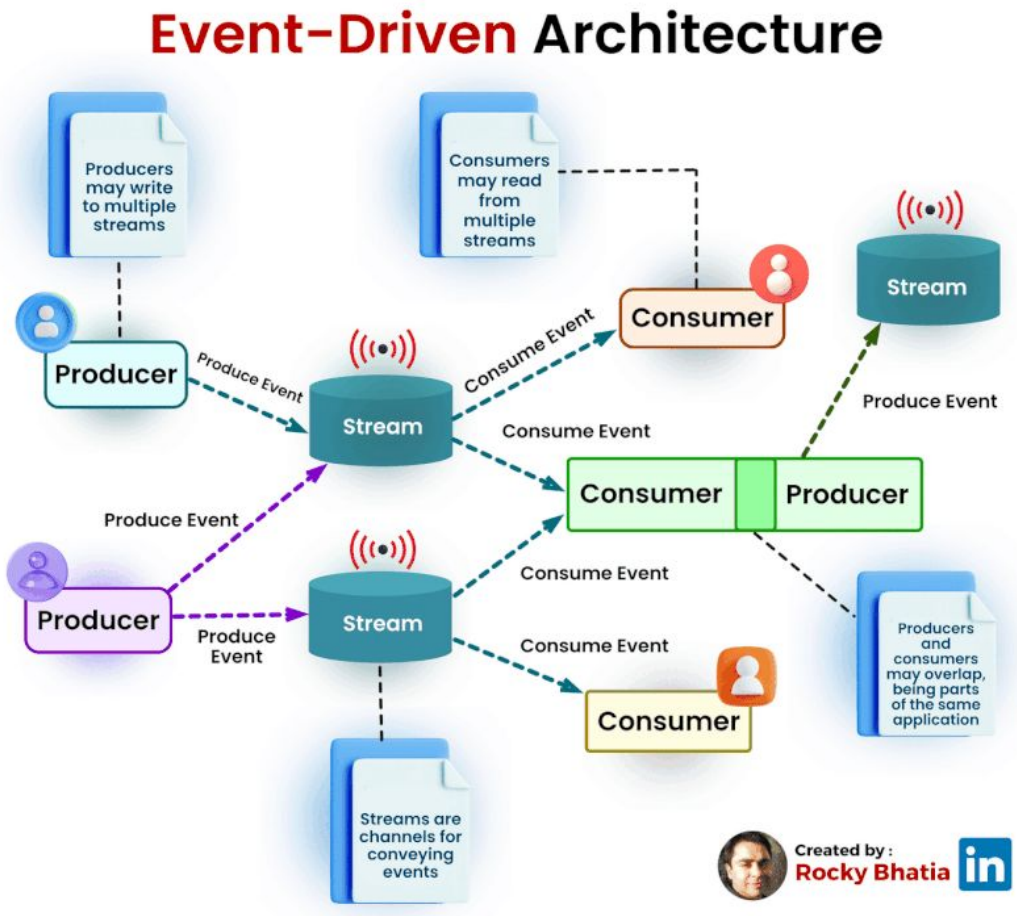
- Event-based APIs
- Limitação do HTTP
- Simulação de comunicação em tempo real com HTTP
- WebSocket
- Funcionamento do WebSocket
- Protocolo STOMP
- Spring WebSocket
- Exemplo
- Exercícios

Event-based APIs

Enquanto o modelo HTTP/REST tradicional se apóia na requisição síncrona e explícita a recursos (Request-Response), a comunicação por WebSockets fundamenta-se em uma arquitetura assíncrona guiada por eventos (Event-Driven).

- Em um **Sistema Baseado em Eventos,** a aplicação reage à ocorrência de fatos ou mudanças de estado (ex: nova mensagem enviada, usuário conectado, notificação emitida) em tempo real.

Event-based APIs



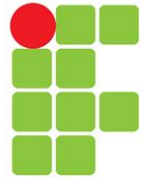
Fonte: https://www.linkedin.com/pulse/event-driven-architecture-rocky-bhatia-k1nuf/

Event-based APIs

Principais Características:

- **Comunicação Reativa e Assíncrona:** O sistema processa e notifica sobre os acontecimentos no exato momento em que ocorrem, sem a necessidade de polling constante ou de manter conexões bloqueadas aguardando respostas;
- **Desacoplamento de Produtores e Consumidores:** Os emissores apenas anunciam que um evento ocorreu, enquanto os manipuladores (handlers) escutam e respondem a esse evento de forma independente;
- **Canal Único Persistente:** Após o estabelecimento inicial da conexão, múltiplos eventos e mensagens fluem continuamente sobre o mesmo canal TCP.

Web API



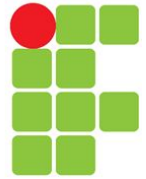
PADRÕES E PARADIGMAS DE API

Event-based APIs

Usados para para compartilhar dados de eventos em tempo real.

- WebHooks
- **WebSockets**
- HTTP Streaming

Limitações do HTTP



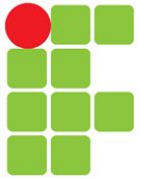
- A arquitetura da Web tradicional (REST / HTTP) foi projetada com base no modelo de requisição e resposta (Request-Response).
- O cliente (navegador ou app) sempre inicia a comunicação enviando uma requisição HTTP para uma URL específica.
- O servidor processa a solicitação, devolve uma resposta (ex: 200 OK) e a conexão TCP subjacente é encerrada. Ou seja, o client não conseguirá mais comunicação por esse conexão.

Limitações do HTTP

Esse modelo apresenta alguns problemas para para Aplicações em Tempo Real:

- **Comunicação Unidirecional**: O servidor é incapaz de enviar dados ativamente ao cliente sem que uma requisição prévia tenha sido feita;
- **Overhead de Cabeçalhos**: Cada nova requisição HTTP carrega cabeçalhos repetitivos (cookies, user-agent, etc.), gerando tráfego desnecessário de rede
- **Latência Elevada**: A necessidade de reabrir ou renegociar conexões inviabiliza interações de altíssima frequência

Limitações do HTTP



Antes do surgimento do WebSocket, os desenvolvedores utilizavam contornamentos sobre o protocolo HTTP para simular atualização em tempo real:

Short Polling (Polling Tradicional):

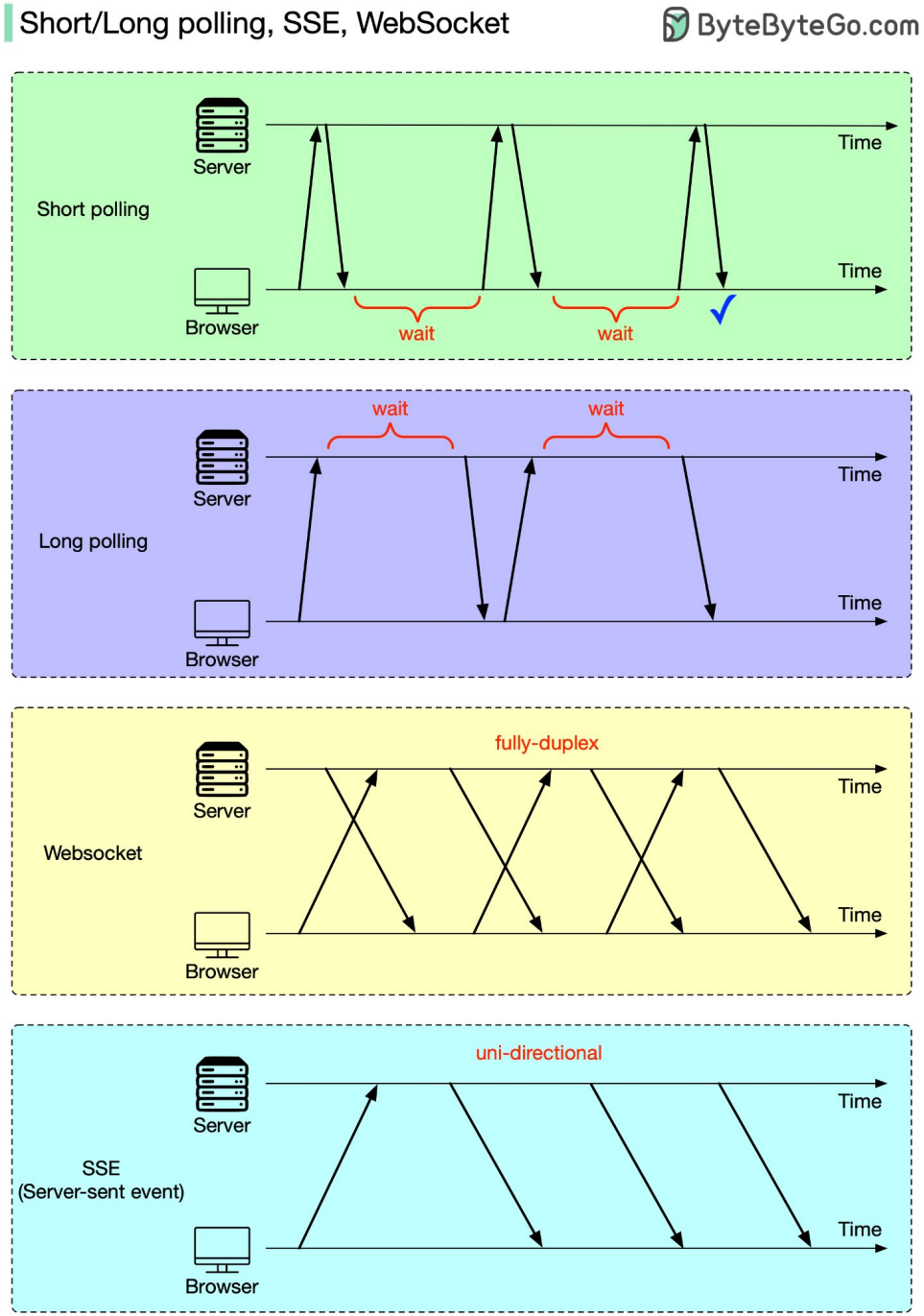
- O cliente faz requisições AJAX periódicas em intervalos fixos (ex: a cada 2 segundos)
- **Desvantagem**: Gera dezenas de requisições vazias quando não há mensagens novas, sobrecarregando o servidor

Long Polling:

- O cliente faz uma requisição e o servidor segura a resposta aberta até que haja novos dados disponíveis
- **Desvantagem**: Complexo de gerenciar no servidor, mantendo **threads bloqueadas** e reatando conexões constantemente

Server-Sent Events (SSE):

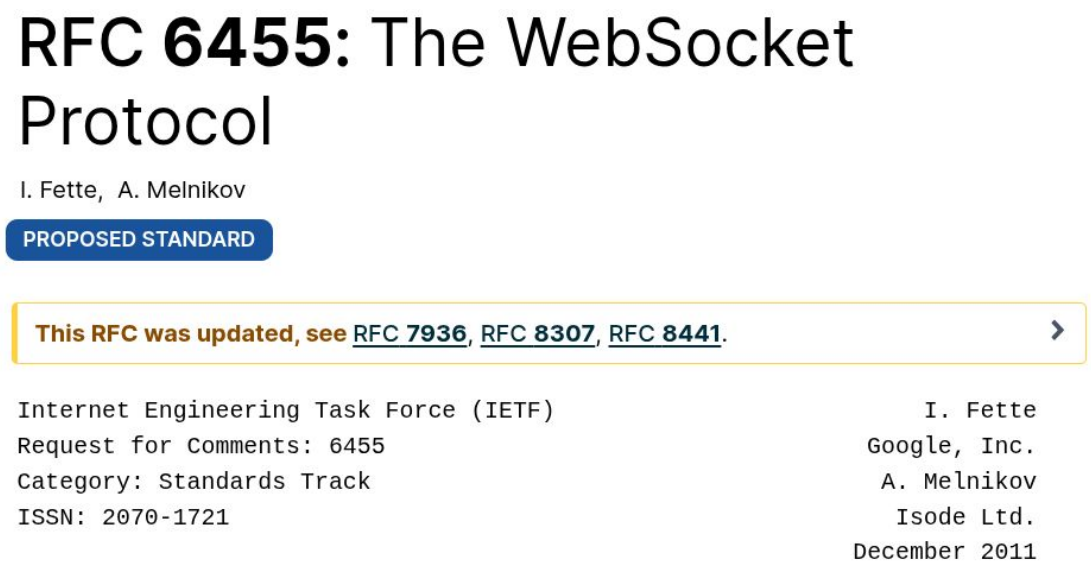
- Permite *streaming* unidirecional (apenas do servidor para o cliente) via HTTP estendido



Fonte: https://assets.bytebytego.com/diagrams/0337-short-long-polling-sse-websocket.jpeg

WebSocket

O WebSocket (padronizado pela RFC 6455) é um protocolo da camada de aplicação que fornece um canal de comunicação **Full-Duplex** sobre uma única conexão TCP duradoura.



RFC **6455**: The WebSocket Protocol

I. Fette, A. Melnikov

PROPOSED STANDARD

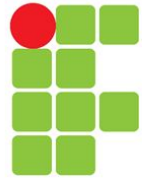
This RFC was updated, see RFC **7936**, RFC **8307**, RFC **8441**.

Internet Engineering Task Force (IETF)
Request for Comments: 6455
Category: Standards Track
ISSN: 2070-1721

I. Fette
Google, Inc.
A. Melnikov
Isode Ltd.
December 2011

https://www.rfc-editor.org/info/rfc6455/

WebSocket



Características:

Full-Duplex: Tanto o cliente quanto o servidor podem enviar e receber dados simultaneamente e de forma independente..

Conexão Persistente: O socket permanece aberto continuamente após ser estabelecido, eliminando a necessidade de abrir novas conexões para cada mensagem.

Mínimo Overhead: Após o aperto de mão (handshake), os dados trafegam envelopados em frames extremamente leves (apenas alguns bytes de cabeçalho).

WebSocket

Comparativo HTTP x WebSocket.

	REST / HTTP	WebSocket
Comunicação	**Unidirecional** (Request-Response)	**Bidirecional** (Full-Duplex)
Ciclo de Vida da Conexão	Fechada após cada resposta (**Stateless**)	**Persistente** (Canal aberto contínuo)
Overhead de Rede	**Alto** (Muitos cabeçalhos repetidos)	**Baixo** (Apenas payloads de dados após o handshake)
Entrega de Dados	Cliente sempre precisa perguntar (**Polling**)	Servidor empurra dados proativamente (**Push**)
Casos de Uso Ideais	CRUDs, integrações estáticas, APIs públicas	Chats, Jogos em tempo real, Dashboards

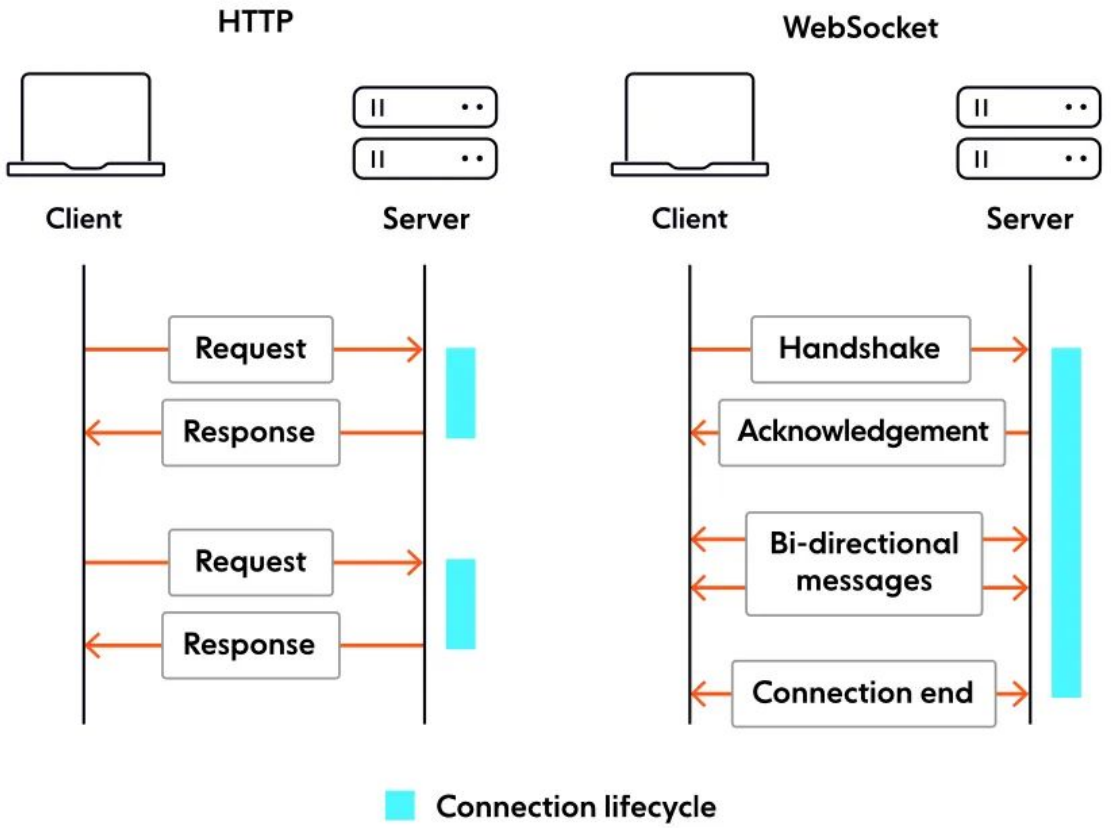


Figure 1.3: WebSockets vs. the traditional HTTP request/response model

Fonte: https://miro.medium.com/0*Y9YDPluI-rKQ0NNb.png

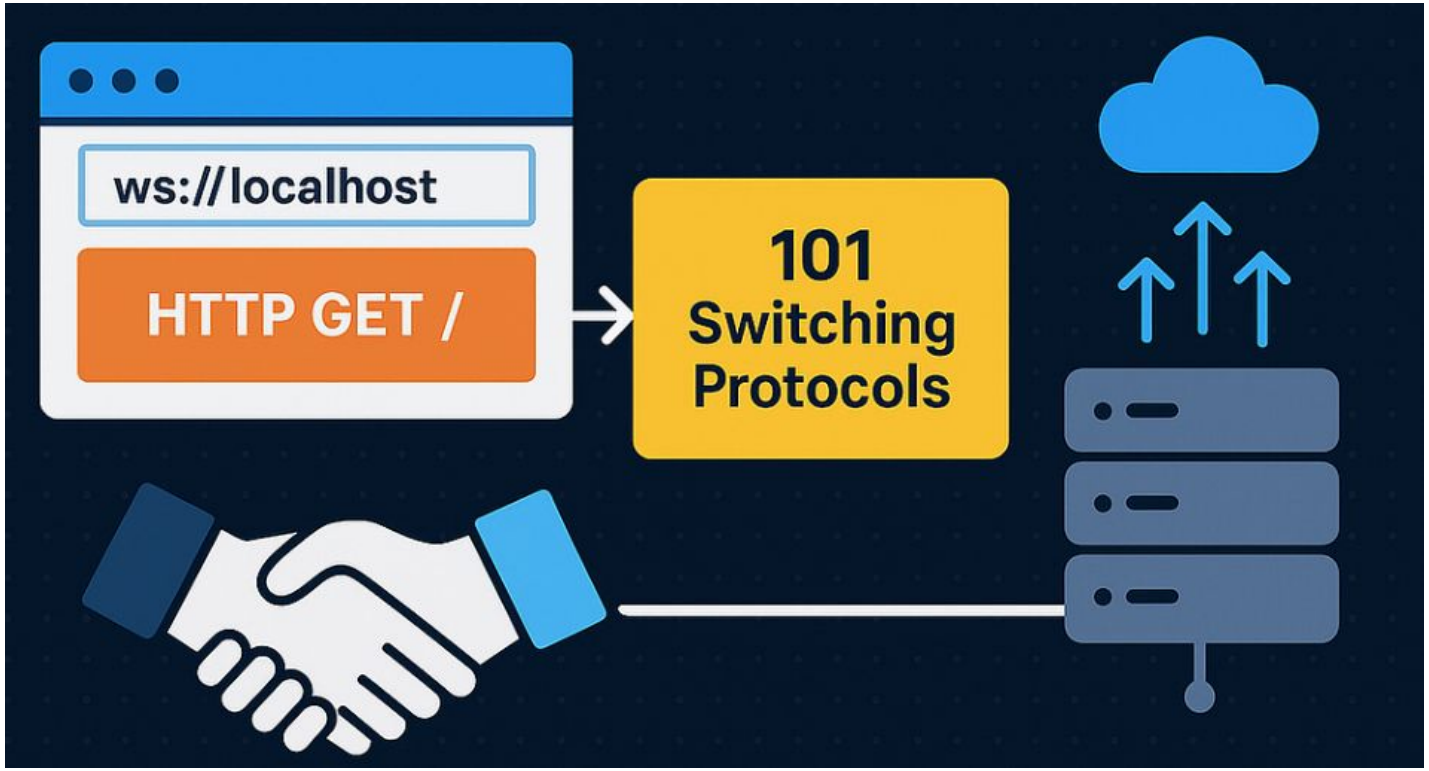
WebSocket

Comparativo com outras técnicas de comunicação em tempo real.

Tecnologia	Iniciador	Bidirecionalidade	Estado da Conexão	Overhead
Polling	Cliente	✖ Falso	Fechada a cada ciclo	Alto
Long-Polling	Cliente	✖ Simulado	Mantida artificialmente	Médio
Server-Sent Events (SSE)	Cliente (Início)	✖ Apenas Servidor -> Cliente	Aberta	Baixo
WebSocket	Ambos	✔ Verdadeiro (Full-Duplex)	Persistente (TCP)	Mínimo (2 bytes)

Funcionamento do WebSocket

Toda conexão WebSocket começa como uma requisição HTTP convencional solicitando uma "**elevação**" de protocolo via *Handshake*.

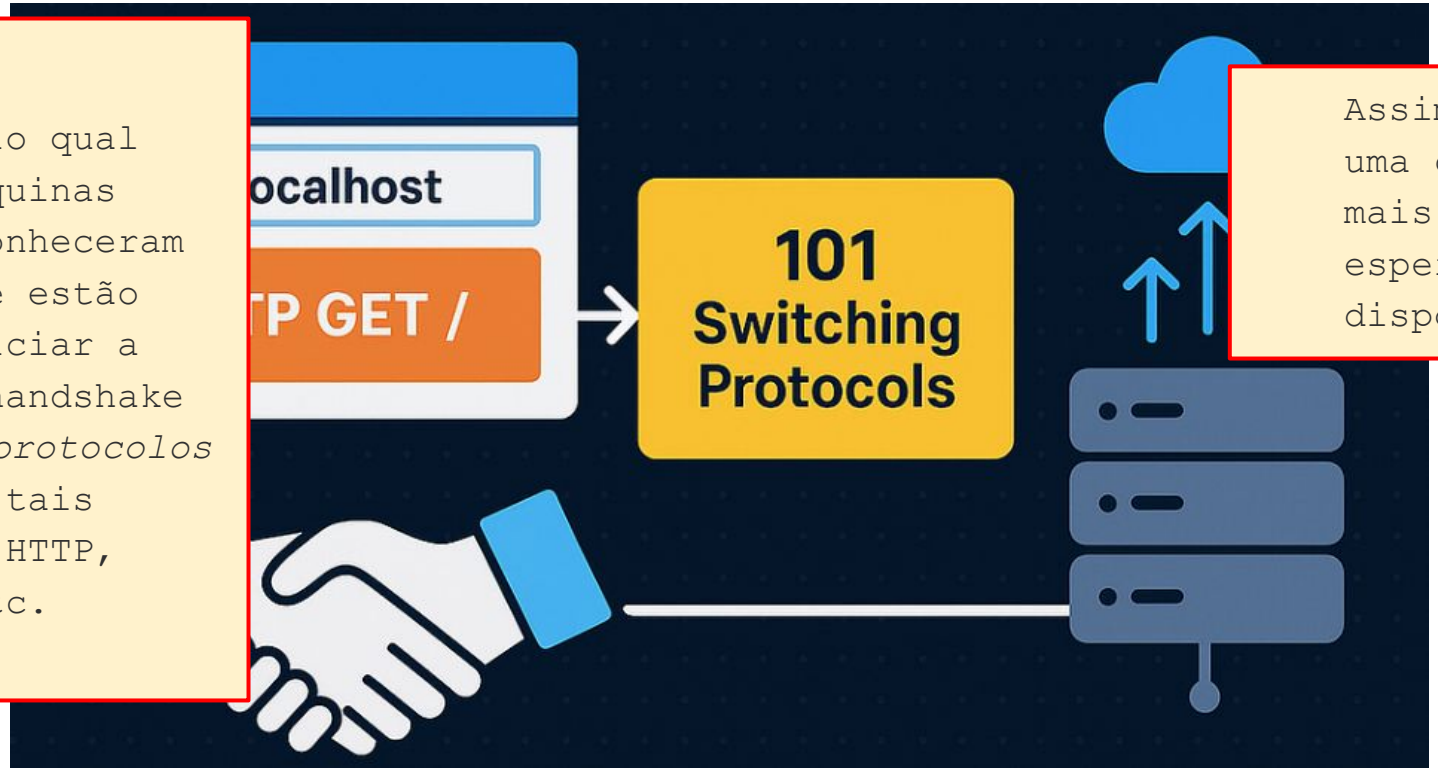


Funcionamento do WebSocket

Toda conexão WebSocket começa como uma requisição HTTP convencional solicitando uma "**elevação**" de protocolo via *Handshake*.

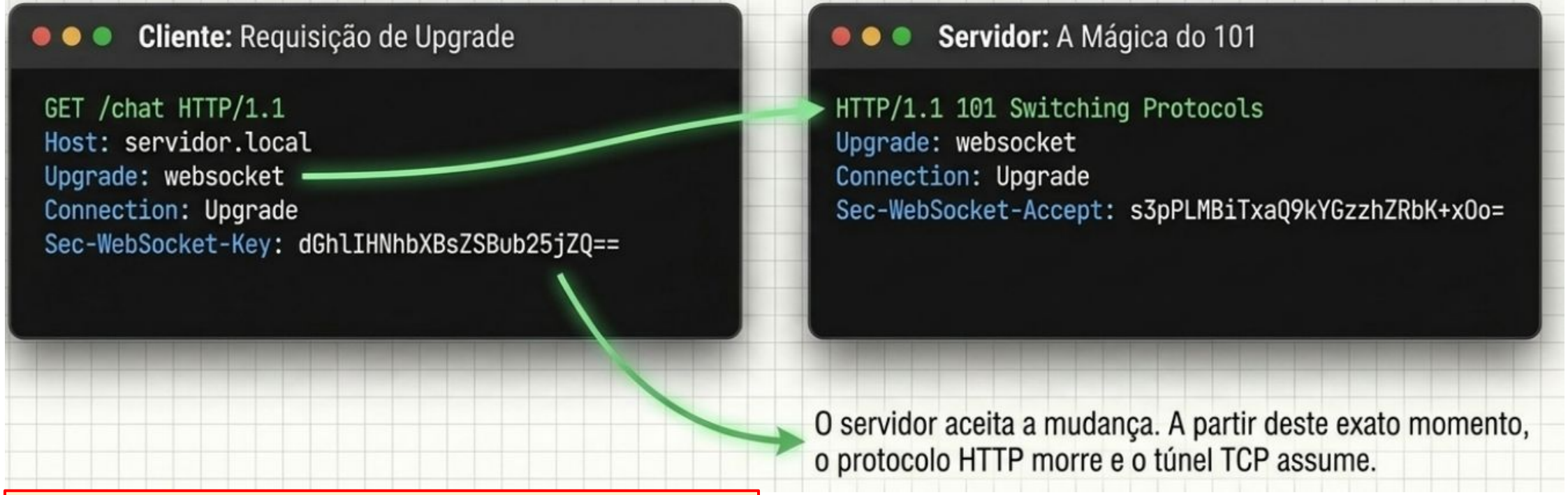
Handshake:

é o processo pelo qual duas ou mais máquinas afirmam que reconheceram umas às outras e estão prontas para iniciar a comunicação. O handshake é utilizado em *protocolos* de comunicação, tais como: FTP, TCP, HTTP, SMB,SMTP,POP3 etc.



Assim você consegue fazer uma conexão entre duas ou mais máquinas só esperando o serviço a ser disponibilizado.

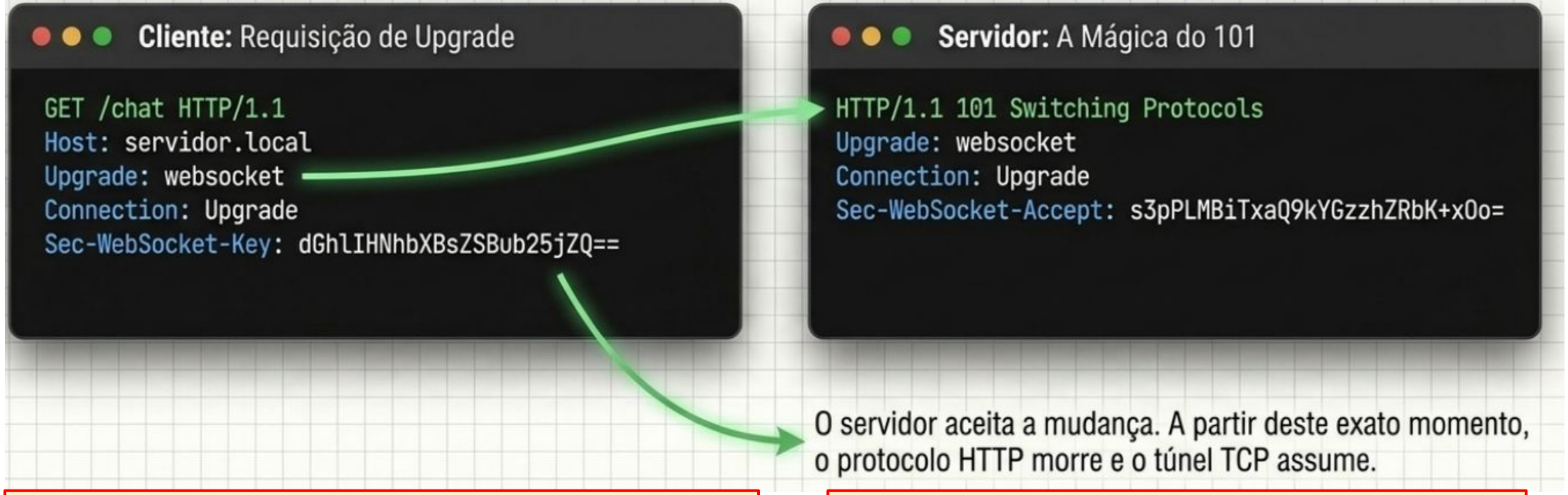
Funcionamento do WebSocket



O cliente envia os cabeçalhos HTTP obrigatórios solicitando a troca de protocolo:

- `Upgrade: websocket`
- `Connection: Upgrade`
- `Sec-WebSocket-Key`: Chave aleatória gerada pelo navegador para validação de segurança

Funcionamento do WebSocket



O cliente envia os cabeçalhos HTTP obrigatórios solicitando a troca de protocolo:

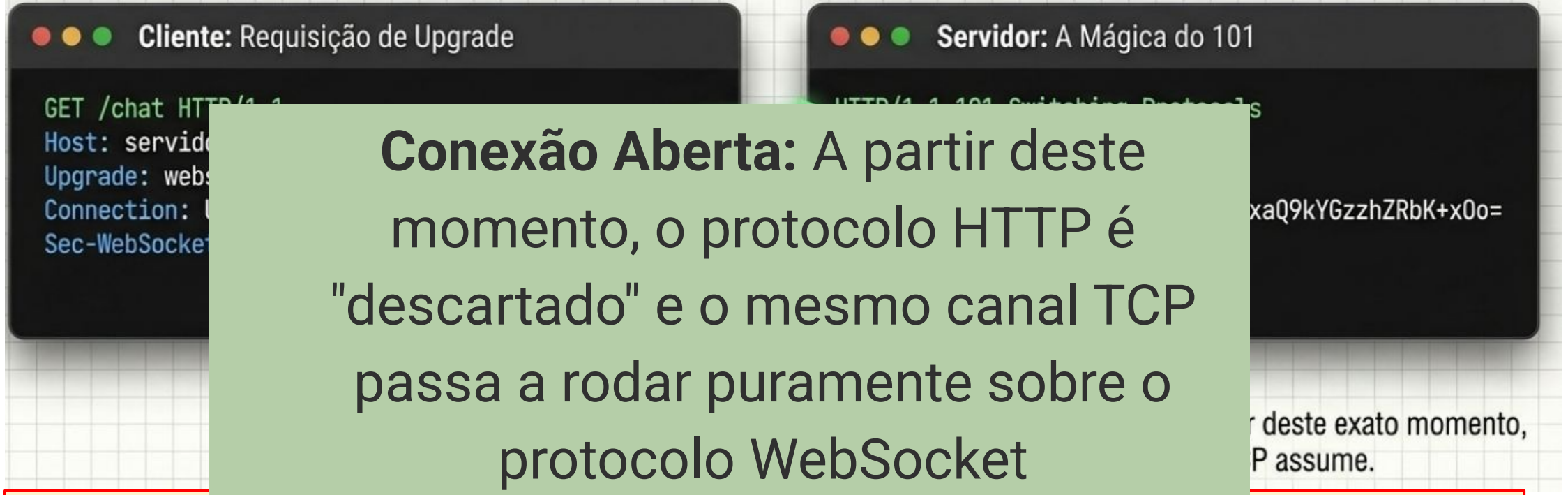
`Upgrade: websocket`
`Connection: Upgrade`
`Sec-WebSocket-Key`: Chave aleatória gerada pelo navegador para validação de segurança

Resposta do Servidor (`101 Switching Protocols`):

Se o servidor aceitar, responde com o código HTTP `101 Switching` Protocols.

Retorna o cabeçalho `Sec-WebSocket-Accept` comprovando o aceite do handshake.

Funcionamento do WebSocket



Conexão Aberta: A partir deste momento, o protocolo HTTP é "descartado" e o mesmo canal TCP passa a rodar puramente sobre o protocolo WebSocket

O cliente envia os cabeçalhos HTTP obrigatórios solicitando a troca de protocolo:

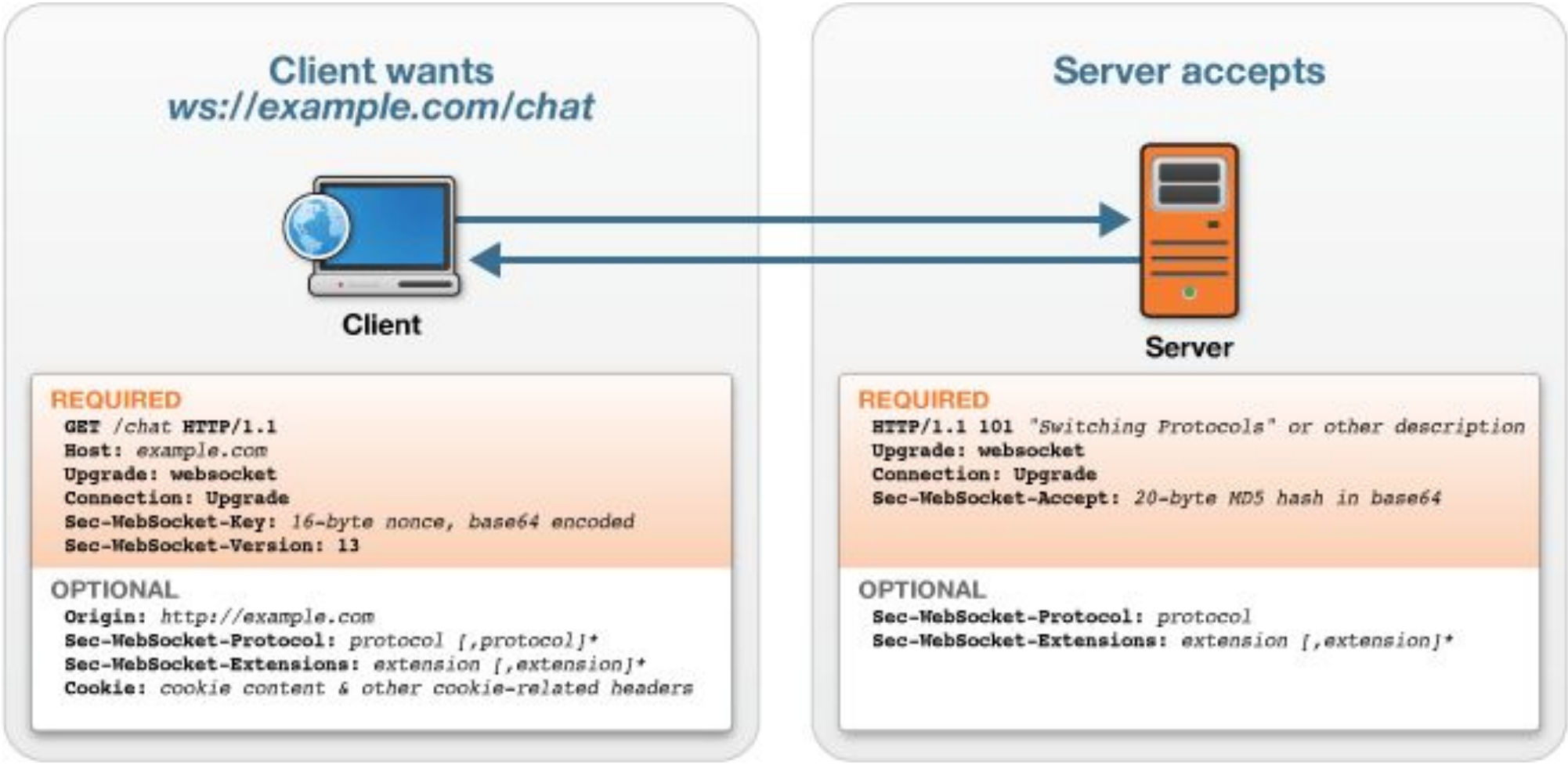
`Upgrade: websocket`
`Connection: Upgrade`
`Sec-WebSocket-Key`: Chave aleatória gerada pelo navegador para validação de segurança

Resposta do Servidor (`101 Switching Protocols`):

Se o servidor aceitar, responde com o código HTTP `101 Switching` Protocols.

Retorna o cabeçalho `Sec-WebSocket-Accept` comprovando o aceite do handshake.

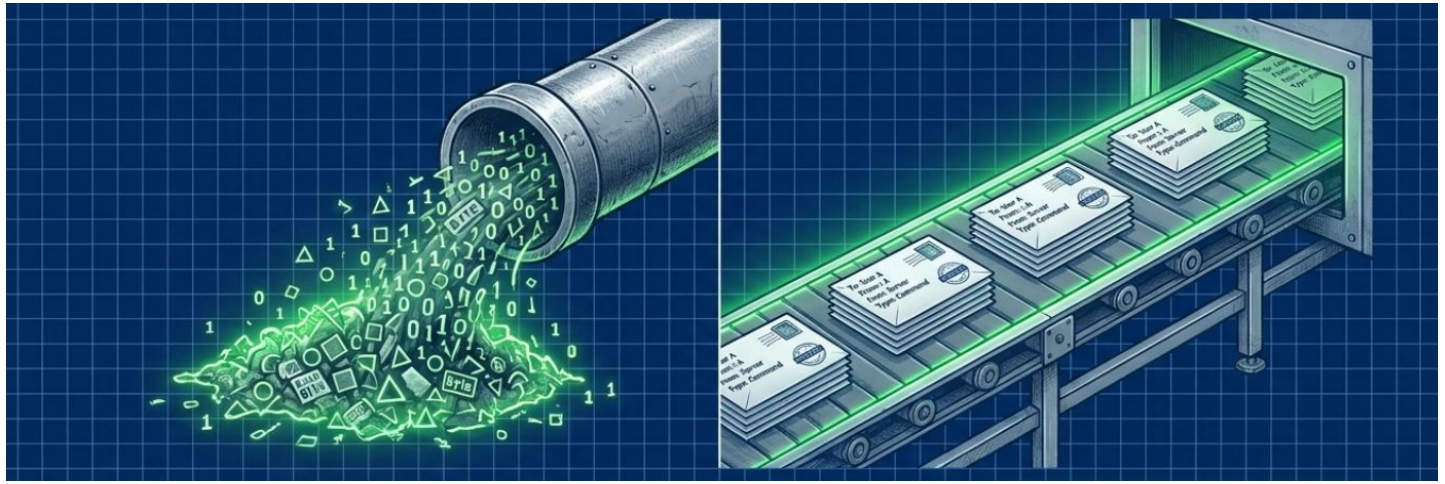
Funcionamento do WebSocket



Fonte: https://atitudereflexiva.wordpress.com/wp-content/uploads/2019/03/websocket_handshake.png

Protocolo STOMP

O WebSocket nativo é um protocolo de baixo nível: ele define como os bytes trafegam, mas não define a semântica da mensagem (não especifica quem envia, para quem vai ou como rotear)



Uma Tela em Branco

O WebSocket fornece a rodovia (TCP), suportando texto e binários, mas sua especificação (RFC 6455) não define o formato interno nem o significado dos dados.

→

Falta de Semântica

Como saber para quem rotear a mensagem? Como o servidor diferencia uma mensagem de chat de um comando de sistema ou um ping de conexão?

→

A Necessidade de um Subprotocolo

Assim como o HTTP deu sentido à web clássica sobre o TCP, precisamos de uma camada superior de regras sobre o WebSocket para estruturar a mensageria.

Protocolo STOMP

Sem um subprotocolo, o desenvolvedor precisaria inventar um formato próprio de mensagem JSON e construir do zero a lógica de roteamento no backend.

STOMP (Simple Text Oriented Messaging Protocol): É um subprotocolo baseado em frames de texto (inspirado no HTTP) negociado durante o handshake



Protocolo STOMP

STOMP segue uma arquitetura publisher-subscriber

O padrão **Publisher-Subscriber (Pub/Sub)** é um modelo de mensageria assíncrona baseado na troca de mensagens orientada a destinos ou tópicos.

Nele, a entidade que envia uma mensagem não precisa conhecer diretamente quem vai recebê-la, promovendo o desacoplamento fraco (*loose coupling*) entre as partes do sistema.

Protocolo STOMP

Os Três Papéis Fundamentais na Arquitetura:

Publisher (Publicador / Produtor):

- É a entidade que gera e envia dados ou eventos para o sistema.
- No protocolo STOMP, envia comandos do tipo SEND contendo o cabeçalho do destino escolhido (ex: /topic/chat).
- Não possui conhecimento sobre a existência ou a quantidade de assinantes conectados.

Subscriber (Assinante / Consumidor):

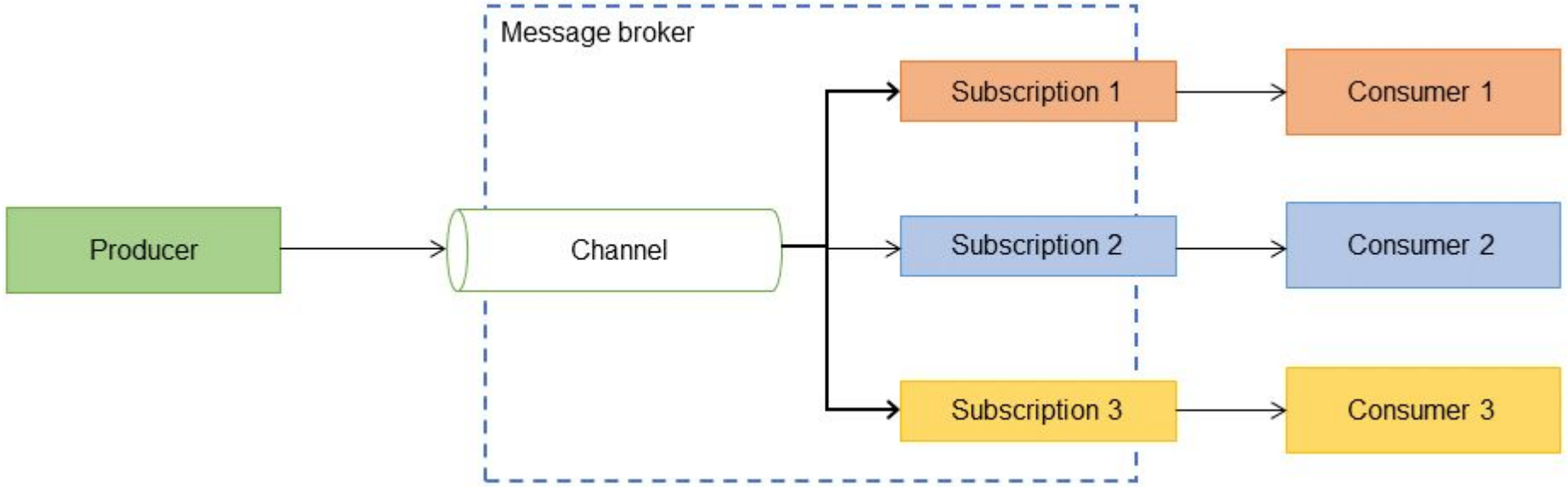
- É a entidade que registra interesse em receber mensagens enviadas para determinado tópico.
- Envia um comando SUBSCRIBE ao servidor indicando o endereço do canal que deseja escutar.
- Recebe as mensagens retransmitidas pelo broker através de comandos MESSAGE.

Message Broker (Intermediário / Roteador):

- É o componente central responsável por gerenciar as inscrições e rotear o tráfego.
- Intercepta as mensagens enviadas pelos *publishers*, identifica os *subscribers* inscritos naquele destino e realiza a transmissão (*broadcast*)

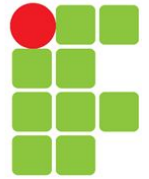
Protocolo STOMP

STOMP segue uma arquitetura publisher-subscriber



Fonte: https://docs.aws.amazon.com/pt_br/prescriptive-guidance/latest/cloud-design-patterns/publish-subscribe.html

Protocolo STOMP

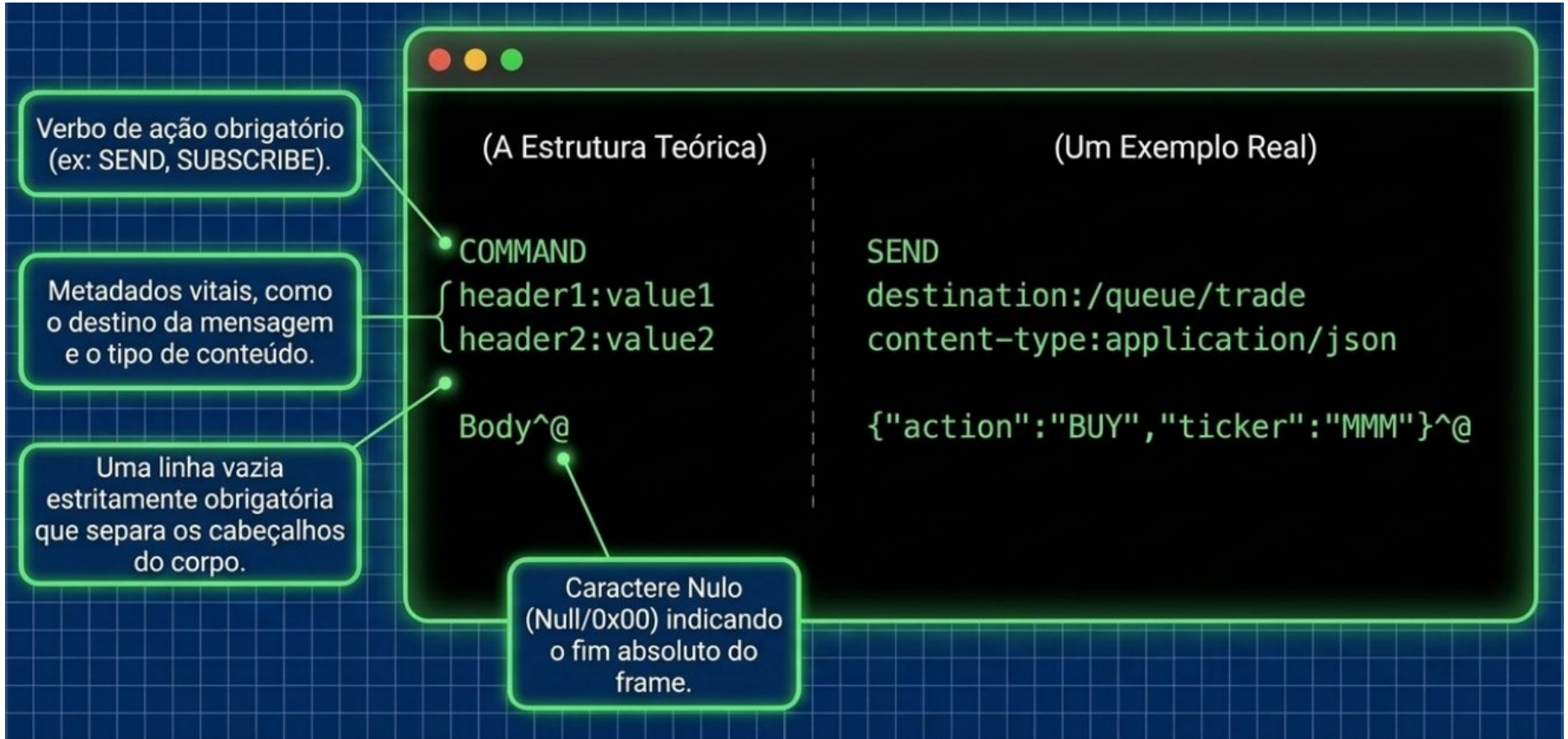


O STOMP utiliza o padrão de projeto **Publish/Subscribe** para desacoplar quem produz a mensagem de quem a consome. Isso é definido por **Comandos (Frame)**

- **Inscrição (SUBSCRIBE):**
  - O cliente informa ao servidor que deseja "ouvir" determinado endereço (ex: /topic/chat).
- **Envio (SEND):**
  - O cliente envia uma mensagem direcionada a um destino (ex: /app/chat.sendMessage).
- **Transmissão (MESSAGE):**
  - O broker distribui a mensagem para todos os clientes inscritos naquele canal.
- **Convenção de Prefixos no Spring Framework:**
  - /topic/ -> Mapeamento **Um-para-Muitos** (*Broadcast* público).
  - /queue/ -> Mapeamento **Ponto-a-Ponto** (Mensagem privada para um único usuário).
  - /app/ -> Mapeamento direcionado aos métodos do **Controller no Spring**.

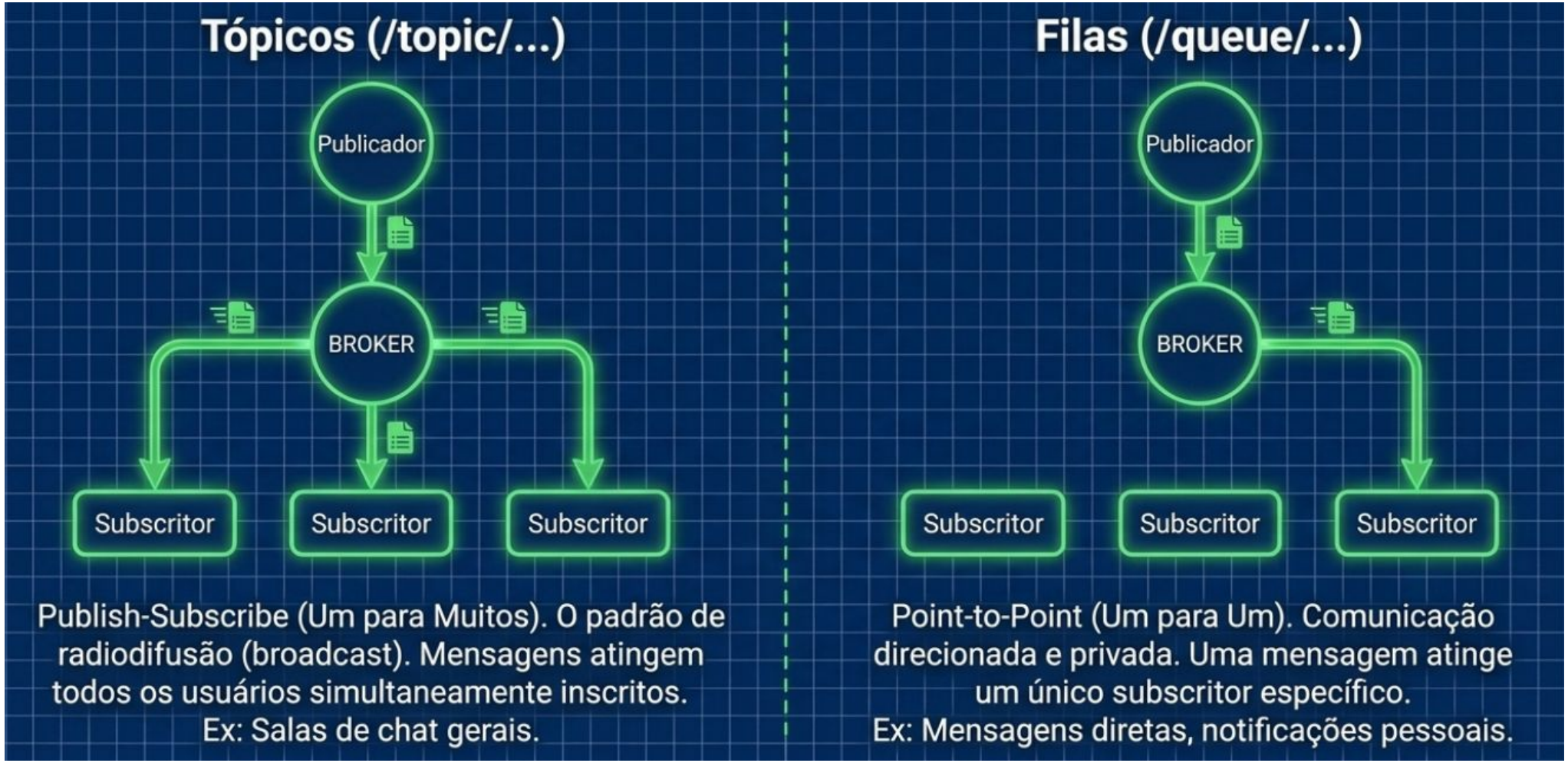
Protocolo STOMP

No WebSocket com **STOMP (Simple Text Oriented Messaging Protocol) os dados enviados são divididos em Frames.**

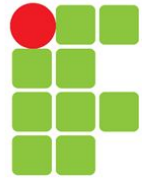


Protocolo STOMP

Convenção de Prefixos



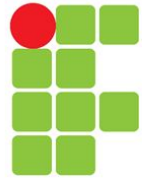
Protocolo STOMP



Opções de Broker:

- **Simple Broker (Padrão):** Um broker interno rodando em memória RAM na própria aplicação Spring Boot (ideal para desenvolvimento e turmas de aula)
- **External Broker (Stomp Broker Relay):** Encaminha as mensagens para um servidor dedicado (RabbitMQ / ActiveMQ / Kafka) via TCP, garantindo alta escalabilidade.

Protocolo STOMP

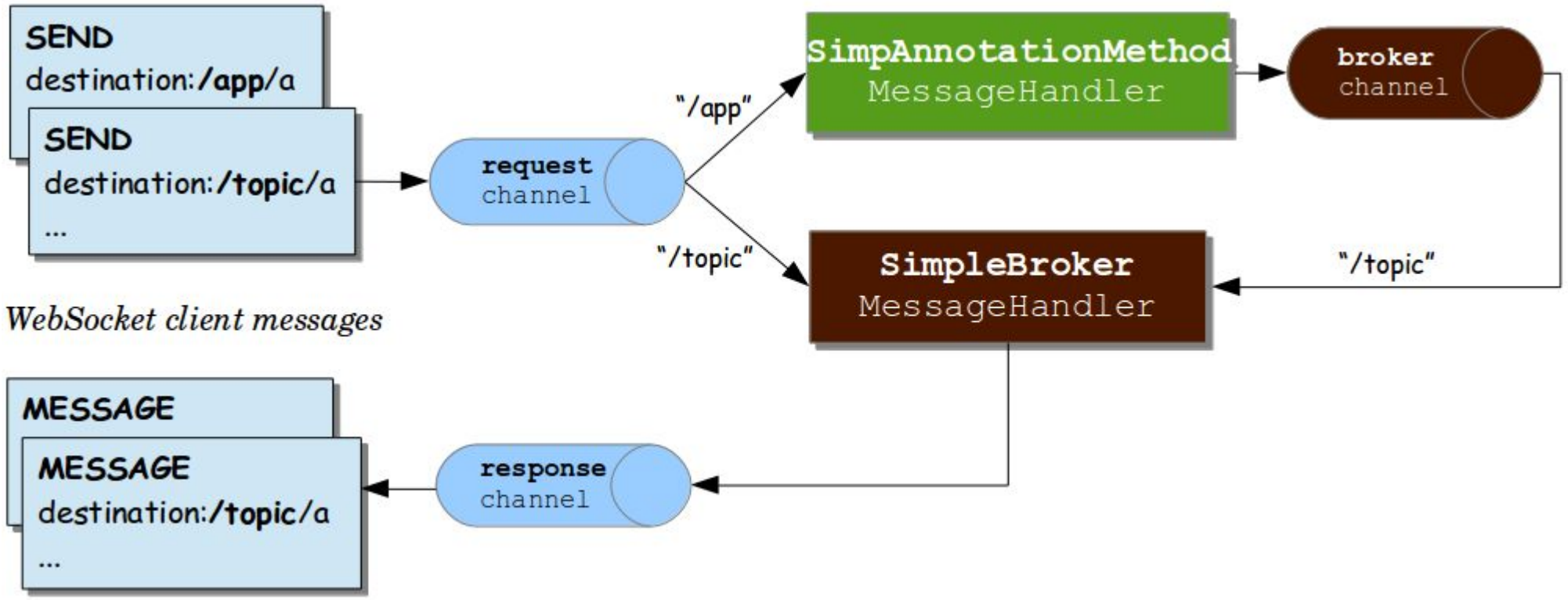


Dentro da aplicação Spring Boot, a mensageria STOMP é processada através de três canais principais de mensagem (Message Channels):

- **clientInboundChannel**: Transporta as mensagens recebidas dos clientes WebSocket para dentro da aplicação
- **brokerChannel**: Transporta mensagens geradas internamente pelos Controllers do Spring até o Message Broker
- **clientOutboundChannel**: Envia as mensagens do Message Broker de volta aos clientes WebSocket conectados

Protocolo STOMP

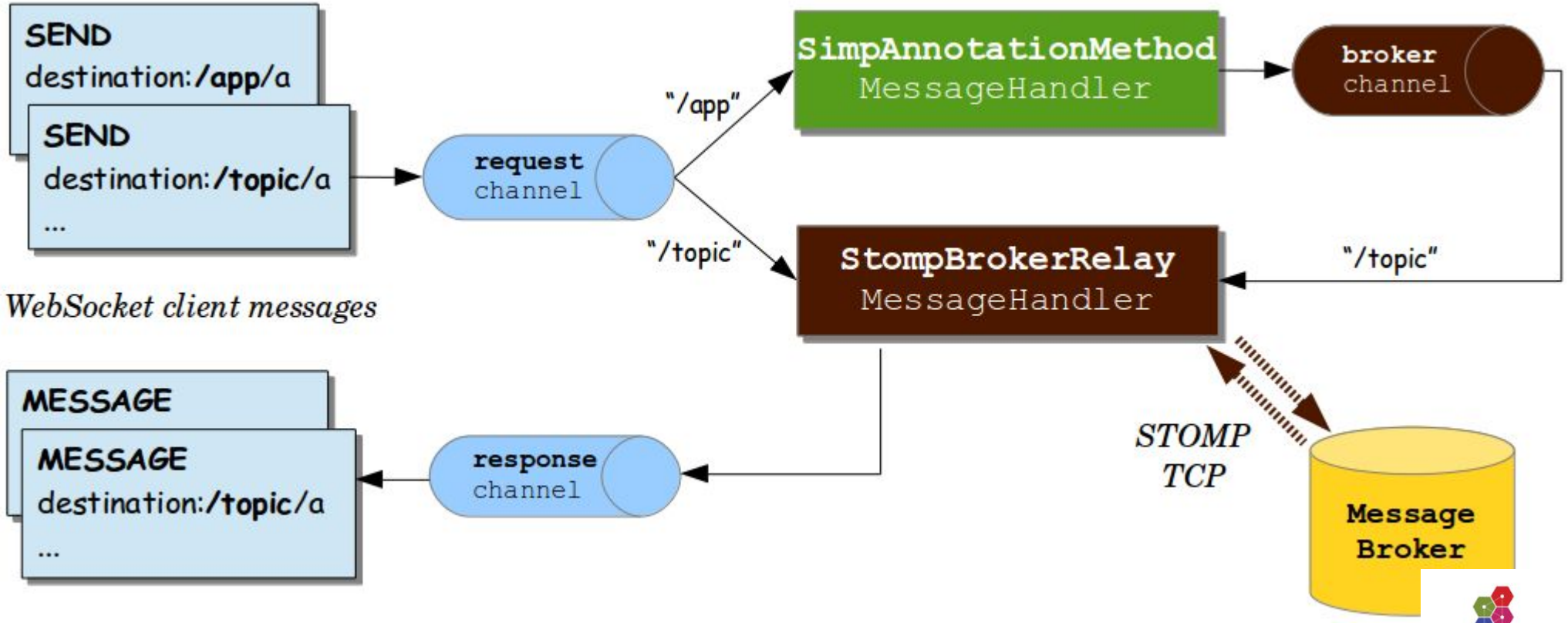
Simple Broker



Fonte:
https://docs.spring.io/spring-framework/reference/web/websocket/stomp/message-flow.html

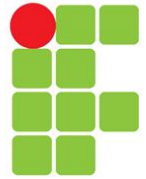
Protocolo STOMP

External Broker



Fonte: https://docs.spring.io/spring-framework/reference/web/websocket/stomp/message-flow.html

Spring WebSocket



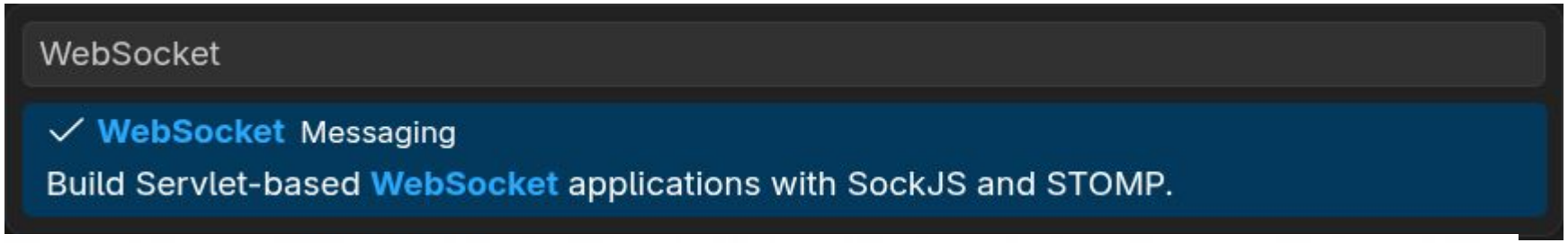
O **Spring WebSocket** é o módulo do Spring Framework (distribuído principalmente nos pacotes ***spring-websocket*** e ***spring-messaging***) que fornece suporte completo para comunicação bidirecional e em tempo real em aplicações Java.

Em vez de exigir que o desenvolvedor lide com conexões TCP de baixo nível ou crie tratadores manuais para cada socket, o Spring oferece abstrações de alto nível que se integram ao ecossistema do Spring Web MVC.

Nos exemplos, vamos utilizar um **Simple Broker (Armazena em memória RAM)**

Spring WebSocket

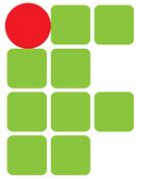
Adicionando ao projeto




```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-websocket</artifactId>
</dependency>
```

Uma única dependência **encapsula todo o núcleo de WebSockets**, habilita o protocolo STOMP e realiza as integrações nativas com o Spring MVC, dispensando a instalação de servidores de mensageria externos para desenvolvimento.

Spring WebSocket



Anotações de Configuração:

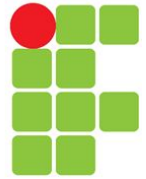
- `@EnableWebSocketMessageBroker`: Habilita o processamento de mensagens baseadas em STOMP e constrói a infraestrutura do Message Broker na aplicação

Classes e Interfaces

- `WebSocketMessageBrokerConfigurer` é uma interface do Spring que define métodos de *callback* para personalizar a configuração do Message Broker e dos pontos de entrada WebSocket.

```
@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer
{
    // Métodos de registro e configuração do broker
}
```

Spring WebSocket



Classes e Interfaces

- A interface `WebSocketMessageBrokerConfigurer` define dois métodos a serem implementados..

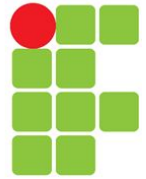
Define como as mensagens serão roteadas entre os clientes e o servidor utilizando a classe de suporte `MessageBrokerRegistry`

configureMessageBroker

```
@Override
public void configureMessageBroker(MessageBrokerRegistry config) {
        // 1. Inscrições e Broadcast (Servidor -> Cliente)
        config.enableSimpleBroker("/topic");

        // 2. Mapeamento de Controllers (Cliente -> Servidor)
        config.setApplicationDestinationPrefixes("/app");
}
```

Spring WebSocket



Classes e Interfaces

- A interface `WebSocketMessageBrokerConfigurer` define dois métodos a serem implementados..

Define como as mensagens serão roteadas entre os clientes e o servidor utilizando a classe de suporte **`MessageBrokerRegistry`**

configureMessageBroker

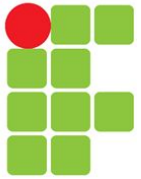
```
@Override
public void configureMessageBroker(MessageBrokerRegistry config) {
        // 1. Inscrições e Broadcast (Servidor -> Cliente)
        config.enableSimpleBroker("/topic");

        // 2. Mapeamento de Controllers (Cliente -> Servidor)
        config.setApplicationDestinationPrefixes("/app");
}
```

Ativa um broker simples em memória RAM. As mensagens enviadas para destinos que começam com **/topic** serão retransmitidas em broadcast para todos os clientes inscritos naquele canal.

Define o prefixo global para mensagens enviadas PELOS clientes que devem ser interceptadas pelos métodos anotados com `@MessageMapping` nos Controllers Spring.

Spring WebSocket



Classes e Interfaces

- A interface `WebSocketMessageBrokerConfigurer` define dois métodos a serem implementados..

`StompEndpointRegistry` para registrar o ponto de contato inicial (URL) onde os clientes solicitarão a conexão WebSocke

registerStompEndpoints

```
@Override
public void registerStompEndpoints(StompEndpointRegistry registry) {
    registry.addEndpoint("/ws")
    .setAllowedOriginPatterns("*") // Config de CORS
    .withSockJS(); // Caso haja bloqueio de WebSocket na rede, usa HTTP
    para simular conexão em tempo real via Polling.
}
```

addEndpoint("/ws"): Mapeia a URL do endpoint HTTP onde o handshake WebSocket será iniciado (ex: http://localhost:8080/ws ou http://<IP_REDE>:8080/ws)

Spring WebSocket



Spring WebSocket

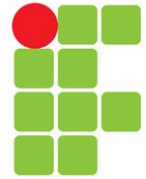
No Spring WebSocket com STOMP, as mensagens recebidas dos clientes não passam por endpoints REST convencionais (**@RestController**), mas sim por Controladores Anotados com **@Controller**.

O Spring intercepta os frames STOMP vindos dos clientes direcionados ao prefixo de aplicação (ex: **/app**) e os roteia para os métodos correspondentes.

Anotações de Roteamento nos Controllers (@Controller):

- **@MessageMapping:** Mapeia destinos de mensagens vindas dos clientes (ex: /app/mensagem para métodos específicos do controlador.
- **@SendTo**: Define o destino/tópico público para onde o valor retornado pelo método deve ser transmitido em *broadcast* (ex: /topic/publico).
- **@SendToUser:** Direciona a mensagem de resposta exclusivamente para a sessão do usuário que enviou a requisição.

Spring WebSocket



```java
@Controller
public class ChatController {
    @MessageMapping("/mensagem")
    @SendTo("/topic/publico")
    public String processarMensagem(String mensagem) {
        return "Servidor recebeu: " + mensagem;
    }
}
```

Atenção: Substitui o tradicional **@RestController**. Aqui o foco não é expor rotas REST estáticas.

Rota de Entrada: O cliente envia dados para `**/app/mensagem**`. Equivalente arquitetônico ao **@PostMapping**.

Rota Automática de Saída: O retorno (String) do método é propagado diretamente para o Broker no canal `**topic/publico**`, atingindo todos os ouvintes inscritos.

Spring WebSocket - Cliente

O backend não trabalha sozinho. O cliente web precisa se adaptar ao protocolo.

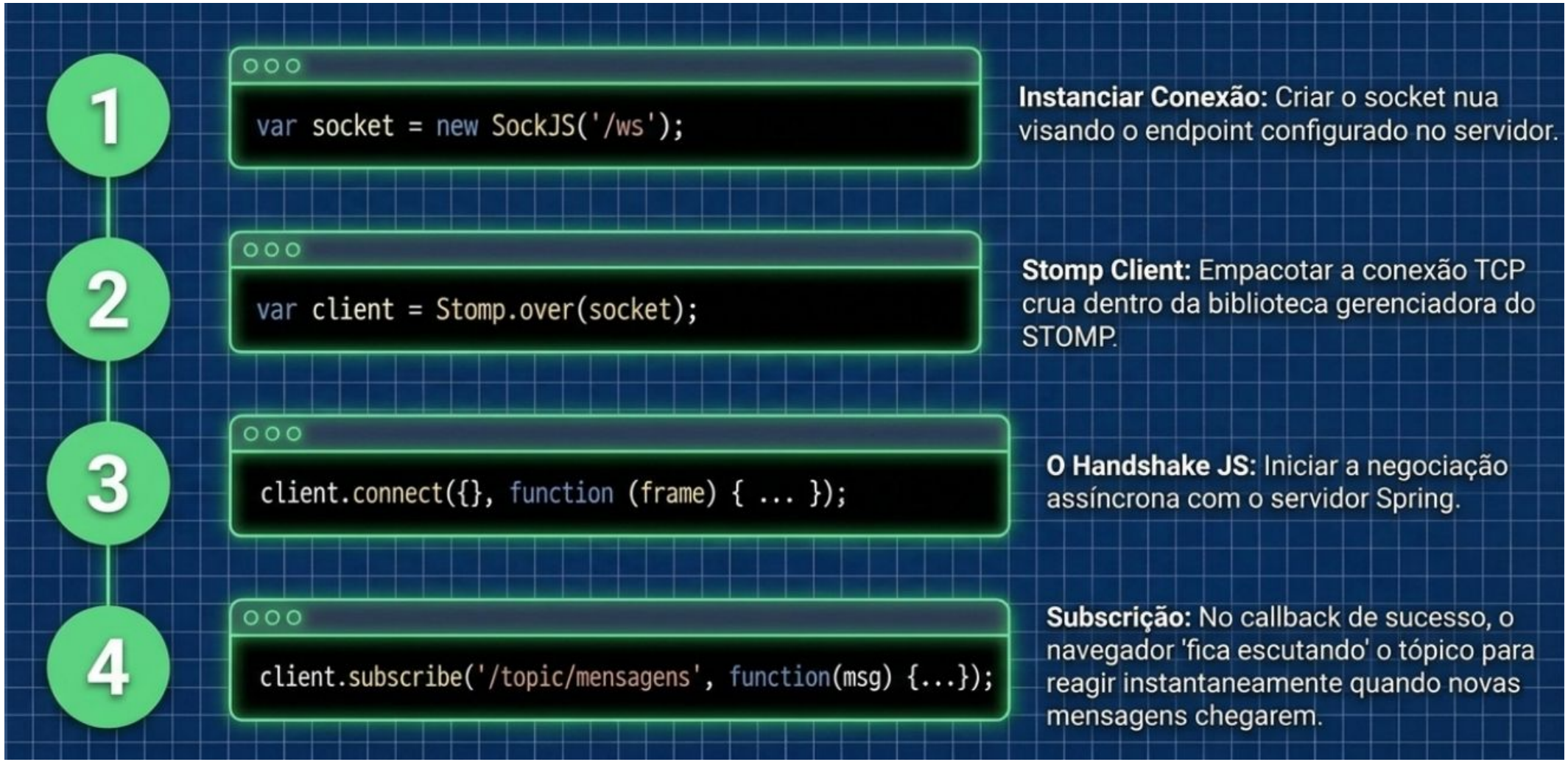


STOMP.js

A **biblioteca cliente** central que entende a arquitetura de mensageria **Pub/Sub**.

Sintaxe semântica: *Utilizada para formatar as mensagens STOMP corretamente e enviar os comandos necessários (`connect()`, `subscribe()` e `send()`) de uma forma que o servidor Spring entenda instantaneamente.*

Spring WebSocket - Cliente

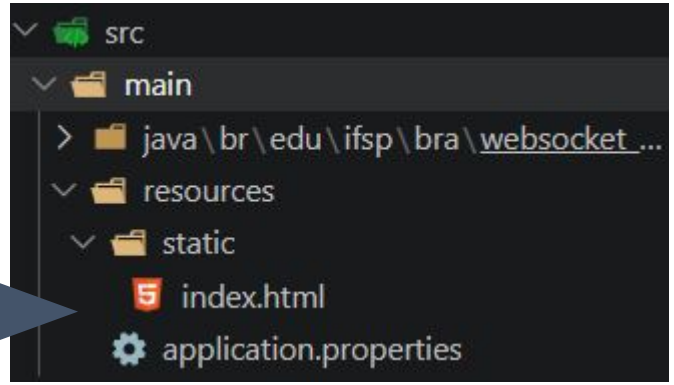


Spring WebSocket - Cliente

Vamos testar utilizando o seguinte HTML.



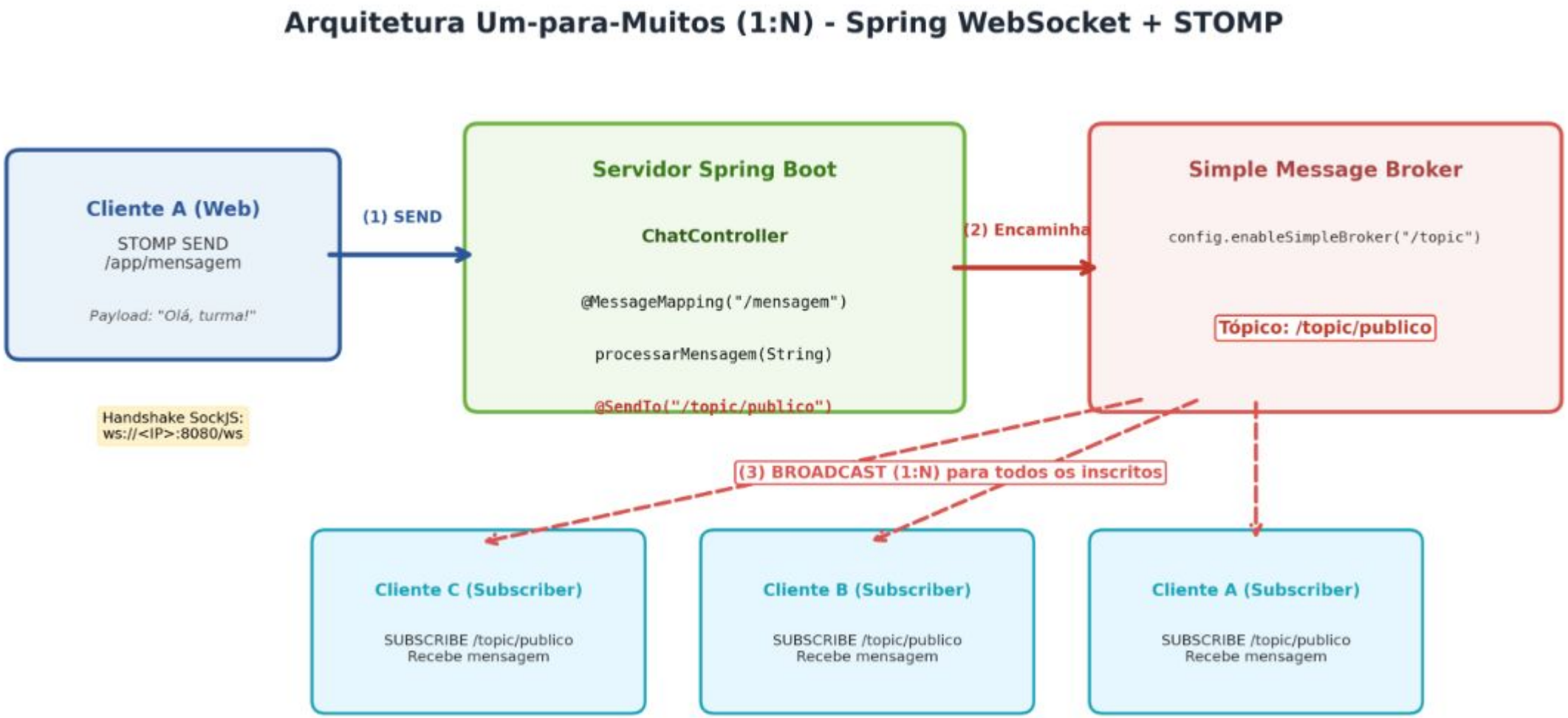
Baixar arquivo index.html.



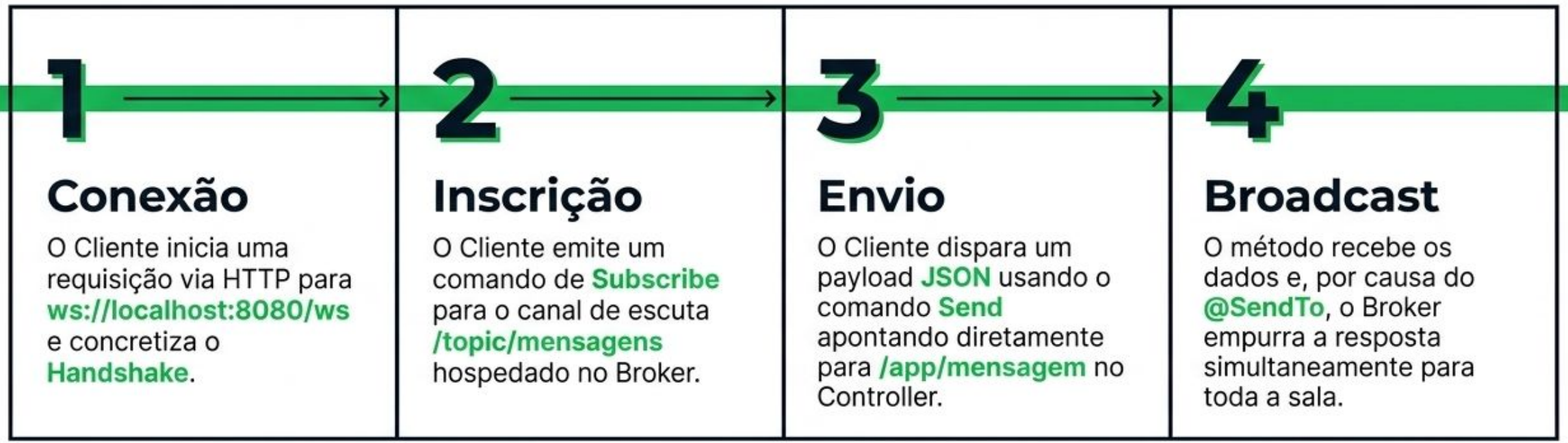
Criar a pasta *static* como na imagem e colar o .html dentro da mesma.

Acesse com http://localhost:8080/index.html

Spring WebSocket - Arquitetura



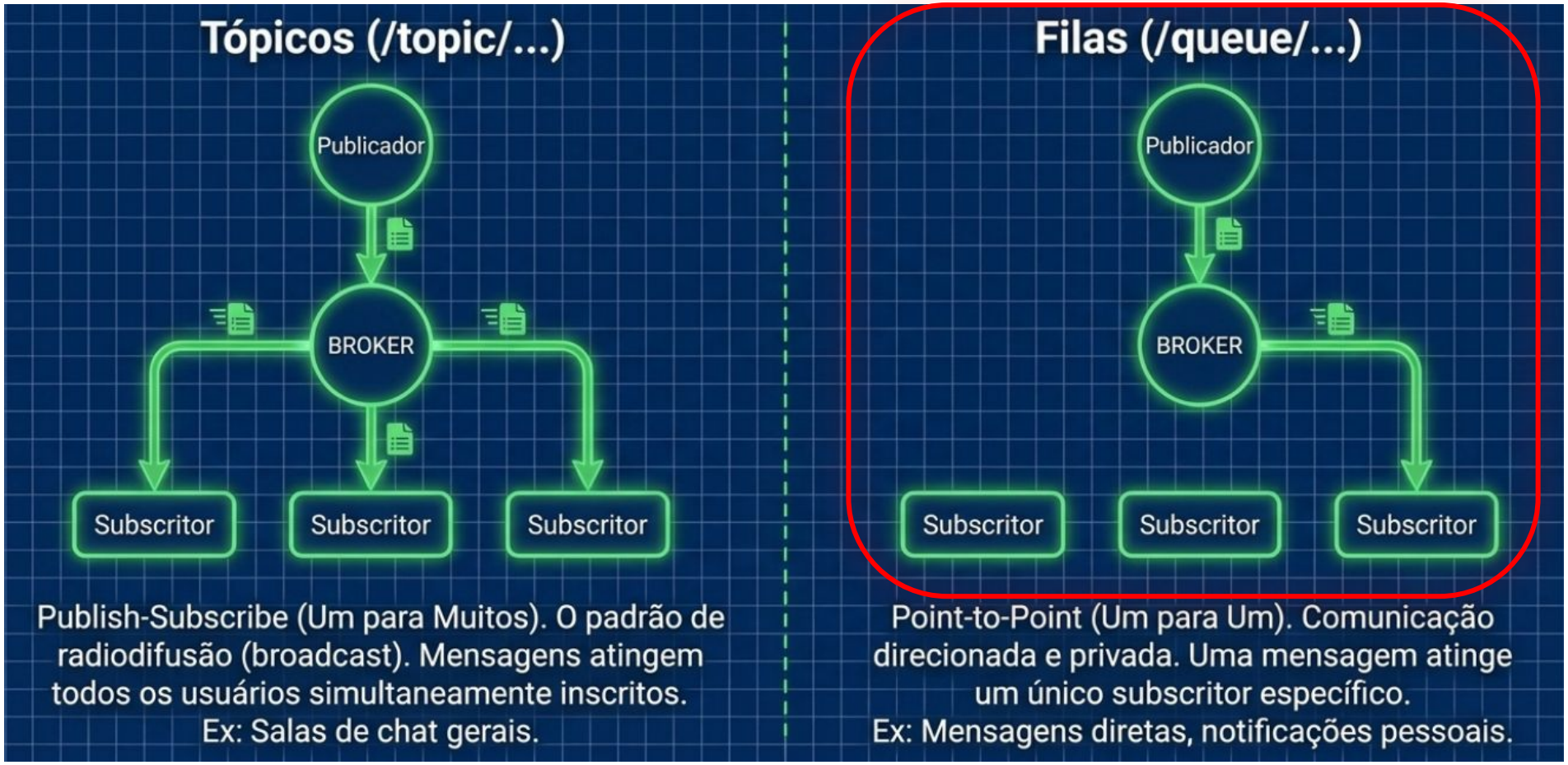
O Fluxo na Prática: Uma Transação Completa



Gemini Notebook

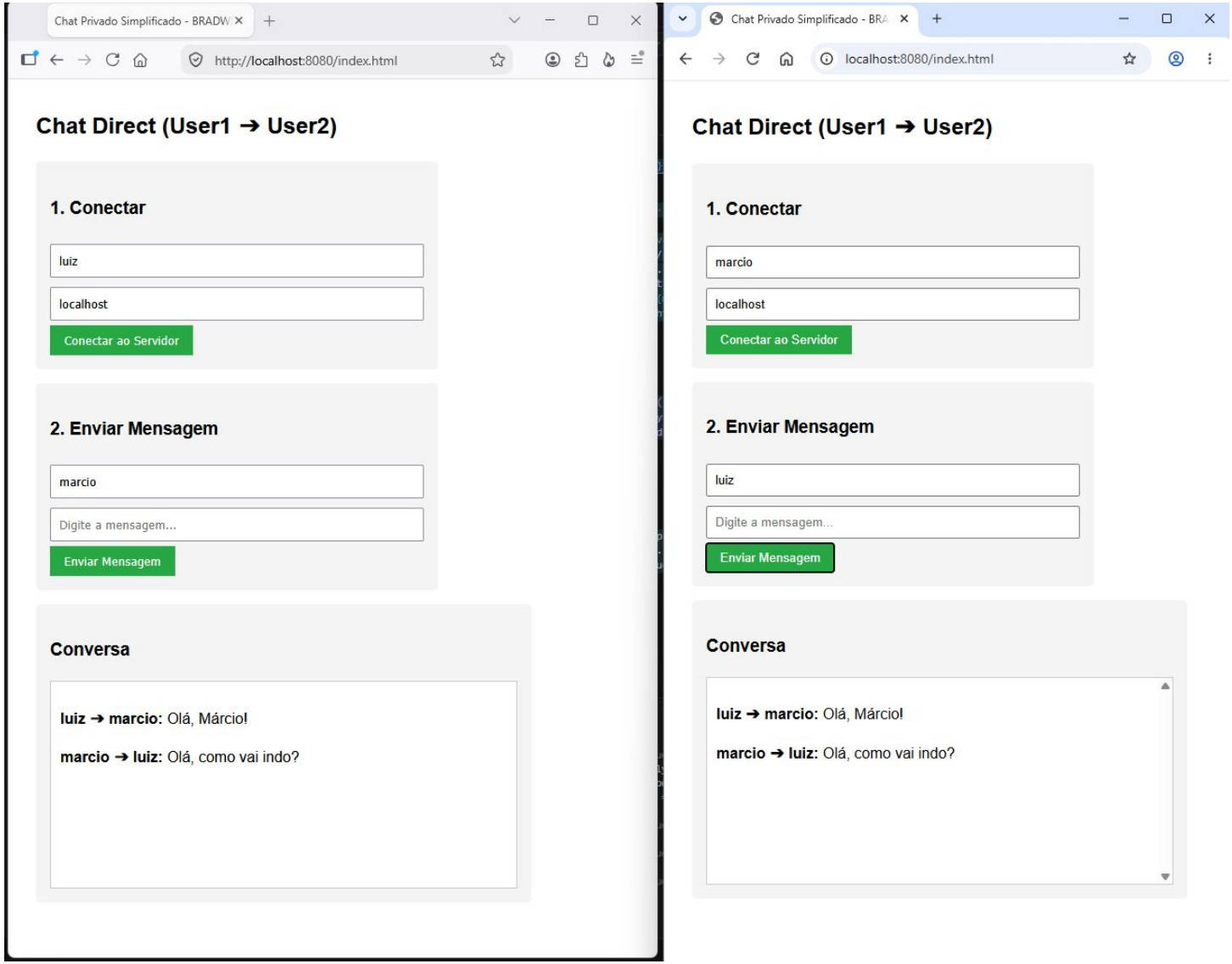
Protocolo STOMP

Implementamos uma estrutura para **tópicos/*topic* (1:N)**. Agora vamos implementar para **fila/*queue* (1:1)**.

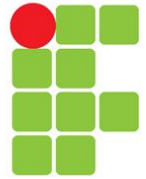


Exercícios

Siga o tutorial disponibilizado no **Moodle** para implementar um **chat** onde um cliente se registra como usuário e digita o nome de outro usuário para conversar em um chat atualizado em tempo real.



Fontes e Links



JIN, Brenda; SAHNI, Saurabh; SHEVAT, Amir. **Designing Web APIs: Building APIs That Developers Love**. " O'Reilly Media, Inc.", 2018.